CSE 310 Online (B2): Mind the Indent

Time: 35 minutes **Full Marks: 10**

Python again. Let's try indents. Indentation does not matter in C, but it matters in python. You will extend the lexical analyzer developed for the offline lexical-analysis assignment. The input will use the existing C-like tokens, but leading spaces will additionally define indentation levels. All previous tokenization, line-counting, scope-table, and error-reporting rules remain unchanged.

Indentation Tokens

At the beginning of every non-empty line, you have to keep track of the leading spaces and compare the space count with the current indentation level.

- If it increases, generate one `<INDENT>` token and store the new level.
- If it decreases, generate one `<DEDENT>` token for every level removed.
- If it is unchanged, generate no indentation token.

The initial indentation level is `0`. At end of file, generate enough `<DEDENT>` tokens to return to level `0`. Indentation spaces themselves produce no ordinary token and are not inserted into the scope table.

Blank lines and lines containing only an existing comment do not affect indentation. You can assume that only spaces are allowed for indentation, no other whitespace character will be used.

For each generated token, add a log entry:

```
Line no <n>: Token <INDENT> found
Line no <n>: Token <DEDENT> found
```

For EOF-generated dedents, use the final line number.

Required error checking

- When indentation decreases, the new width must match a level already stored in the stack. Otherwise, report:

```
Line no <n>: Inconsistent dedentation
```

Stop scanning after the first indentation error. Count it in the existing total error count.

Hints

- You have to use a stack.
- The amount of space characters in the beginning of a line is an important attribute to track.

Sample I/O

Only the additional indentation-related token and log entries are shown. Existing output is omitted.

Sample 1: Valid nested indentation

Input

```
if(x)
    y = 1;
    if(z)
        y = 2;
    y = 3;
return 0;
```

Additional token-file content

```
<INDENT> <INDENT> <DEDENT> <DEDENT>
```

Additional log-file content

```
Line no 2: Token <INDENT> found
Line no 4: Token <INDENT> found
Line no 5: Token <DEDENT> found
Line no 6: Token <DEDENT> found
```

Sample 2: Multiple dedents and an indentation error

Input

```
while(x)
        if(y)
            x = x + 1;
    y = 0;
return 0;
```

Indentation width on line 4 does not match any previous level.

Additional token-file content

```
<INDENT> <INDENT>
```

Additional log-file content

```
Line no 2: Token <INDENT> found
Line no 3: Token <INDENT> found
Line no 4: Inconsistent dedentation
```

No `DEDENT` token is generated for the invalid line because scanning stops at the first indentation error.

Marks Distribution

- Detecting indentation at the beginning of non-empty lines: 2 marks
- Stack management and correct `INDENT` generation: 2 marks
- Correct `DEDENT` and EOF handling: 2 marks
- Ignoring blank and comment-only lines: 1 mark
- Required error detection: 3 marks

Submission: Submit only your `2205ABC.l` file. Do not submit `lex.yy.c`, executables, token files, or log files.